

AUTONOMOUS AGENTS · SERIES TWO

The Scholar Series

Lesson 3 — Reflection & Self-Critique.

The agent stops trusting its first draft and becomes its own toughest editor.

★ The centerpiece — and still built in both frameworks

Where we are. Our agent loops autonomously (Lesson 1) and plans its own work before executing (Lesson 2). But it accepts its first draft without question — it never reads its own output to ask whether it's any good. This lesson teaches the most powerful idea in the series: *reflection*. The agent will learn to critique its own work, find its weaknesses, and revise — the loop that separates a mediocre agent from an excellent one.

What this lesson covers

- 1 **The scene** — a complete-looking draft that's quietly weak
- 2 **The concept** — why an agent can improve work by judging it
- 3 **The architecture** — the generate-critique-revise loop
- 4 **Build it in LangGraph** — a critic node that loops back to the writer
- 5 **Build it in CrewAI** — a critic agent, and the guardrail alternative
- 6 **Compare** — explicit critic loop vs automatic guardrail retry
- 7 **Run it + the new crack** — and why long work needs progress memory

PART 1

The scene: a draft that looks done but isn't

Lesson 2's agent plans well and covers every part of the goal. It produces a draft that, at a glance, looks finished. But read it the way a demanding editor would, and the cracks show.

Goal: Brief me on the current state of solid-state batteries.

Draft (Lesson 2 agent): "...Manufacturing is the main challenge. Several companies are making progress. Solid-state batteries will likely be commercially significant soon, transforming the industry."

It reads smoothly, but a careful editor immediately objects. "The main challenge" — which challenge, specifically? "Several companies" — name them. "Will likely be commercially significant soon" — by when, and on what evidence? "Transforming the industry" — that's a vague flourish, not a finding. The draft has the *shape* of a good briefing but is thin on specifics, unsupported in its claims, and padded with empty phrases. And the Lesson 2 agent has no idea, because it never re-read its own work. It wrote the draft and declared victory.

THE FAILURE, PRECISELY

The agent treats its first draft as its final answer. It has no mechanism to step back, read what it wrote with a critical eye, and notice that claims are vague or unsupported. A first draft is rarely anyone's best work — human or machine — yet our agent ships it unexamined. What's missing is *reflection*: the act of judging one's own output.

PART 2

The concept: improving work by judging it

Reflection is, to my mind, the most quietly powerful idea in all of agent design, and here is why it works. A language model is often a better *critic* than it is a first-pass *author*. Asked to write a briefing in one go, it produces something decent but unremarkable. But asked to read that same briefing and point out its weaknesses, it can be genuinely incisive — it will spot the vague claims, the missing evidence, the empty phrases. Reflection harnesses this by splitting the work into two roles: a *generator* that drafts, and a *critic* that finds the flaws, with the generator then revising based on the critique.

This mirrors exactly how good human writing happens. You rarely write something excellent in a single pass. You write a draft, then you switch hats and read it critically — "this paragraph is weak, this claim needs a source, this sentence says nothing" — and then you revise. Often you loop: draft, critique,

revise, critique again, until it's genuinely good. The critic and the author are the same person wearing different hats, and the switching between them is where quality comes from. We're teaching the agent to wear both hats.

The reason this is such a leap in capability is that it lets the agent *exceed its own first attempt*. Every capability so far — looping, planning — helped the agent gather and organise. Reflection helps it improve *quality*, which is a different and harder thing. An agent that can critique and revise its own work is no longer limited to what it can produce in one shot; it can iterate toward something better, the way a skilled writer does.

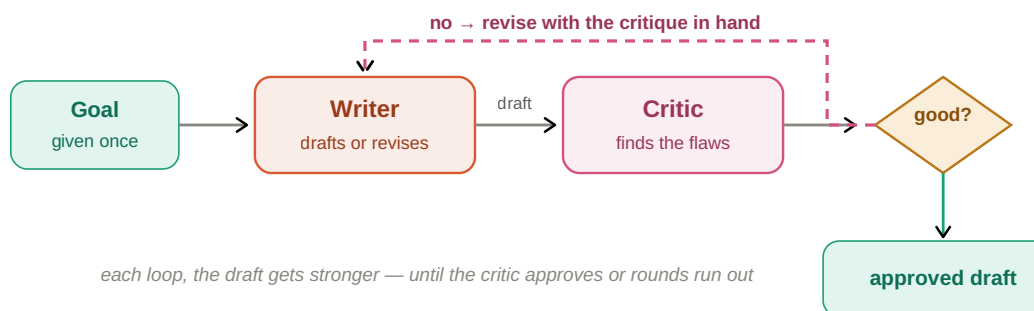
THE ONE IDEA OF THIS LESSON

Reflection splits work into a generator that drafts and a critic that finds flaws, then revises based on the critique — often looping until the work is good. It exploits a key truth: models are frequently better at critiquing than at first-pass authoring. Reflection is how an agent exceeds its own first attempt and reaches real quality.

PART 3

The architecture: generate, critique, revise

The architecture adds a new loop, distinct from Lesson 1's research loop. After the agent generates a draft, a critic examines it and produces feedback. If the critique finds real problems, the draft goes back for revision; if the work passes, it exits. This generate-critique-revise cycle is the engine of self-improvement.



The reflection loop. The critic's verdict is the decision point: real flaws send the draft back to the writer for revision; a clean pass exits. The loop is what lets quality climb with each round.

The key feature is the critic feeding back into the writer. Unlike a one-way pipeline, the draft can cycle through writer and critic several times, improving on each pass, until the critic is satisfied or a round limit is reached. Note that round limit — just as Lesson 1's loop needed an exit, this one does too, or the agent could polish forever. Getting that balance right, between enough revision and endless revision, is part of the craft.

PART 4

Build it in LangGraph: a critic node loops to the writer

In LangGraph, reflection is two nodes — a writer and a critic — with a conditional edge from the critic that either loops back to the writer or exits. Because LangGraph is explicit, you can watch the draft bounce between them and see the critique drive each revision.

```
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

class DraftState(TypedDict):
    goal: str
    draft: str          # the current draft
    critique: str        # the critic's latest feedback
    rounds: int          # revision rounds so far (the exit guard)
```

Why this exists: the state holds both the **draft** and the **critique** so they can pass between writer and critic, plus a **rounds** guard so revision can't loop forever.

```
def writer(state: DraftState) -> DraftState:
    # Write a draft, OR revise using the critic's feedback if we have it.
    instruction = "Write a briefing on the goal."
    if state["critique"]:
        instruction = f"Revise your draft using this critique:\n{state['critique']}"
    state["draft"] = llm.invoke([SystemMessage(content=instruction),
                                HumanMessage(content=f"Goal: {state['goal']}\nCurrent draft: {state['draft']}")]).content
    state["rounds"] += 1
    return state

def critic(state: DraftState) -> DraftState:
    # Read the draft critically. Say "APPROVED" only if genuinely good.
    state["critique"] = llm.invoke([
        SystemMessage(content="You are a demanding editor. List specific "
                               "weaknesses: vague claims, missing evidence, empty phrases. "
                               "If the draft is genuinely strong, reply exactly 'APPROVED'."),
```

```
HumanMessage(content=state["draft"])]).content
return state
```

Why this exists: these are the two hats. **writer** drafts or revises; **critic** hunts for flaws. The critic's instruction to be demanding is what makes reflection improve quality rather than rubber-stamp.

```
def verdict(state: DraftState) -> str:
    # Exit if approved or we've revised enough; else loop back to revise.
    if "APPROVED" in state["critique"] or state["rounds"] >= 3:
        return "done"
    return "revise"

graph = StateGraph(DraftState)
graph.add_node("writer", writer)
graph.add_node("critic", critic)
graph.set_entry_point("writer")
graph.add_edge("writer", "critic")
# THE REFLECTION LOOP: critic sends the draft back to the writer to revise.
graph.add_conditional_edges("critic", verdict,
                           {"revise": "writer", "done": END})

agent = graph.compile()
```

Why this exists: the conditional edge from **critic** back to **writer** is the reflection loop made visible. "revise" returns to the writer with the critique in hand; "done" exits. You can see the quality cycle in the wiring.

PART 5

Build it in CrewAI: a critic agent, plus the guardrail shortcut

CrewAI offers two ways to reflect, and showing both is genuinely instructive. The first mirrors LangGraph in spirit: a separate critic agent reviews the writer's task. The second is CrewAI's own idiomatic feature, the *guardrail*, which folds critique-and-retry into a single task automatically.

First, the explicit critic-agent approach, which keeps the critique visible as its own step:

```
from crewai import Agent, Task, Crew, Process

writer = Agent(role="Writer", goal="Draft a briefing on the topic",
               backstory="You write clear first drafts.")
editor = Agent(role="Demanding Editor",
               goal="Find every weakness and push for a stronger revision",
               backstory="You never accept vague claims or unsupported statements.")

draft_task = Task(description="Draft the briefing.",
                  expected_output="A first-draft briefing.", agent=writer)
revise_task = Task(
```

```
description="Critique the draft, then produce a revised, stronger version.",
expected_output="A revised briefing addressing all weaknesses.",
agent=editor, context=[draft_task]) # the draft flows to the editor
```

```
crew = Crew(agents=[writer, editor], tasks=[draft_task, revise_task],
            process=Process.sequential)
```

Why this exists: this is the two-hat pattern as two agents. The editor receives the writer's draft via `context` and produces the improved version — reflection you can read at each step.

Second, CrewAI's idiomatic shortcut — a *guardrail*, which automatically critiques a task's output and forces a retry until it passes, hiding the loop entirely:

```
# A guardrail: CrewAI checks the output against this rule and auto-retries
# the agent until it passes. The critique-and-revise loop is hidden.
quality_task = Task(
    description="Write the briefing.",
    expected_output="A briefing on solid-state batteries.",
    agent=writer,
    guardrail="Reject if any claim is vague or lacks specific evidence; "
              "require named companies and a dated timeline.",
    guardrail_max_retries=3, # the built-in revision limit
)
```

Why this exists: the string `guardrail` becomes an automatic LLM critic. CrewAI keeps retrying the task until the output satisfies the rule or the retry limit is hit — reflection, with zero loop code.

PART 6

Compare: visible loop vs automatic guardrail

This lesson gives the richest comparison yet, because the two frameworks now embody two genuinely different attitudes toward reflection: make it explicit and inspectable, or make it automatic and invisible.

LANGGRAPH VS CREWAI — REFLECTION

	LangGraph	CrewAI
The critique	A visible node output you can read each round	Critic agent: visible · Guardrail: hidden
The loop	You wire the critic-to-writer edge yourself	Sequential agents, or auto-retry via guardrail
Control	Full — shape the exit, inspect every revision	Guardrail trades control for one-line simplicity
Effort	More code, total transparency	Guardrail is the least code of any approach
Best when	You must audit how quality improved	You want a quality bar enforced with minimal code

The honest verdict, and a genuinely useful one for real projects: if you need to see how the work improved — to log each critique, to debug why a revision went a certain way, to tune the exit condition — LangGraph's explicit two-node loop is the clearer choice, because every step is an inspectable object. If you simply want to enforce a quality bar and don't need to watch the sausage being made, CrewAI's guardrail is remarkably elegant: a single line that turns a task into a self-critiquing, self-retrying one. There's even a middle path in CrewAI's explicit critic-agent approach, which keeps the critique visible while staying in the role-based style. The judgement, as always, is how much visibility the job actually requires.

PART 7

Run it, and the new crack

Run the reflective agent on the weak draft from Part 1 and watch quality climb with each round.

Round 1 draft: "...several companies are making progress... significant soon."

[critic] Vague. Which companies? "Soon" is not a date. "Transforming" is filler. Add specifics and evidence.

Round 2 revision: "...Toyota and QuantumScape target pilot production by the late 2020s; the central hurdle is manufacturing defect-free solid electrolyte layers at scale..."

[critic] APPROVED — claims are now specific and supported.

The transformation is real and visible. The critic caught exactly the weaknesses a human editor would, and the revision addressed them with named companies, a dated timeline, and a specific hurdle. The agent produced work better than its own first attempt — the hallmark of reflection. But now consider what happens as Scholar takes on a *longer* assignment, one that runs for many planning, research, and reflection steps across a large report. Somewhere around the fifteenth step, a subtle problem appears:

the agent starts to lose the thread. It half-forgets what it already covered, or revisits a section it already finalised, because everything — the plan, the findings, the drafts, the critiques — is piling up and the agent has no organised sense of *where it is* in the overall job.

THE PROBLEM LESSON 4 WILL FIX

Reflection makes each piece better, but across a long, multi-step assignment the agent loses track of its own progress — what's done, what's in draft, what remains. In Lesson 4 we'll give it *progress memory*: a structured, working record of the state of the whole job, distinct from the customer-memory of series one. The agent is about to gain a sense of where it stands in a long piece of work.

End of Scholar Lesson 3 — the centerpiece. You taught the agent to critique and revise its own work, in two frameworks, and saw the difference between an explicit reflection loop and an automatic guardrail. Next: helping the agent keep track of a long job.